

It is excellent that you provided the exact database migration scripts you executed alongside the source code. This context is exactly what is needed to accurately weave the new schema logic (like `is_deleted`, `is_featured`, and `updated_at`) directly into the active routing logic without disrupting the frontend configuration.

Here is the detailed diagnostic breakdown of the duplications and conflicting logic present in the uploaded file, followed by the fully rectified code.

1. Duplicated Initialization & Middleware

The Issue: The entire initialization block (Express app, PostgreSQL pool, Cloudinary config, Brevo SDK, and global/auth rate limiters) was declared twice. This causes memory leaks, duplicate connection pools, and immediate crashing upon startup due to re-declaring `const` variables. **The Fix:** I have stripped out the redundant declarations. The server now boots a single instance of Express, establishes a single robust connection pool to Neon PostgreSQL, and instantiates the middleware pipeline strictly once.

2. The `is_deleted` Column Synchronization (Soft Deletes)

The Issue: Your database script successfully added the `is_deleted` column to all 16 tables, but the first half of your code was still querying `SELECT * FROM table`. This means deleted items would still show up on the admin panel. **The Fix:** I have injected `WHERE is_deleted = FALSE` into every single GET request (Users, Services, Sub-services, Portfolio, Blogs, Media, Invoices, Consultations, etc.). This ensures your frontend admin panel respects the database's soft-delete status.

3. Testimonial `is_featured` & Consultation Alignments

The Issue: The second half of your file contained upgraded PUT routes for Testimonials and Consultations that utilized the newly added database columns (`is_featured` and `updated_at`). However, because they were at the bottom, they conflicted with the original routes. **The Fix:** I merged the upgraded parameters into the primary routing module. The Testimonials endpoint now securely accepts and updates both status and `is_featured`, completely aligning with your database schema. I also preserved the legacy `/approve` route just in case your frontend is currently relying on it.

4. Email Broadcasting Crash Prevention

The Issue: In Module 9 (Newsletter), the first half of your code attempted to broadcast using `transporter.sendMail()`. However, `transporter` (Nodemailer) was never initialized, which would cause a fatal crash on execution. The second half of your code had the rectified Brevo SDK implementation utilizing a BCC Privacy Shield. **The Fix:** I removed the broken Nodemailer route and placed the fully functioning Brevo broadcasting pipeline into Module 9. (I kept `require('nodemailer')` at the top to strictly honor your preservation requirements, even though Brevo is now handling the heavy lifting).

5. Advanced Query Engine Route Conflict

The Issue: There were three different GET `/api/orders` endpoints. Two basic ones, and an

upgraded one utilizing the buildAdvancedQuery function at the bottom of the file. **The Fix:** I consolidated these into a single, powerful flow. The buildAdvancedQuery function acts as the primary driver for the Orders ledger, and defaults to is_deleted = FALSE, fulfilling both the historical data mapping and the new soft-delete parameters.

Fully Rectified server.js Code

```
//
=====
=
// DAN74TECH MEDIA - UNIFIED BACKEND SERVER PLATFORM (server.js)
//
=====
=

require('dotenv').config();
const express = require('express');
const { Pool } = require('pg');
const path = require('path');
const cors = require('cors');
const multer = require('multer');
const fs = require('fs');
const cloudinary = require('cloudinary').v2;
const streamifier = require('streamifier');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');
const nodemailer = require('nodemailer'); // Retained per explicit preservation requirements
const PDFDocument = require('pdfkit');
const Brevo = require('@getbrevo/brevo');

// Initialize Express App Engine
const app = express();
const PORT = process.env.PORT || 10000;
const JWT_SECRET = process.env.JWT_SECRET ||
'dan74tech_media_secure_jwt_core_token_secret_key';

// Initialize Neon PostgreSQL Database Engine Pool
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: { rejectUnauthorized: false }
});

pool.query("SELECT NOW()")
  .then(() => console.log("✅ Postgres Database Connected Successfully"))
  .catch(err => console.error("❌ Database Connection Error:", err));
```

```

// Configure Cloudinary Integration
cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME || 'dan74tech',
  api_key: process.env.CLOUDINARY_API_KEY || "",
  api_secret: process.env.CLOUDINARY_API_SECRET || ""
});

// Initialize Brevo Client
let defaultClient = Brevo.ApiClient.instance;
let apiKey = defaultClient.authentications['api-key'];
apiKey.apiKey = process.env.BREVO_API_KEY;
const brevoEmailInstance = new Brevo.TransactionalEmailsApi();

// ===== MIDDLEWARE CONFIGURATION =====
app.use(helmet({ contentSecurityPolicy: false }));
app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(express.static(path.join(__dirname)));

// Global API Request Rate Limiter Node
const globalLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 1000,
  message: { error: "Too many corporate operational requests from this endpoint, please retry later." }
});
app.use('/api/', globalLimiter);

// Auth Specific Security Rate Limiter
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 100,
  message: { error: "Security threshold reached. Verification authentication requests throttled." }
});
app.use('/api/auth/', authLimiter);

// ===== UPLOADS LOCAL FALLBACK STORAGE SYSTEM =====
const uploadDir = path.join(__dirname, 'uploads');
if (!fs.existsSync(uploadDir)) {
  fs.mkdirSync(uploadDir, { recursive: true });
}
app.use('/uploads', express.static(uploadDir));

const localStorage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, uploadDir);
  }
});

```

```

    },
    filename: (req, file, cb) => {
        const uniqueName = 'portfolio-' + Date.now() + path.extname(file.originalname);
        cb(null, uniqueName);
    }
});
const uploadLocal = multer({ storage: localStorage });

const memoryStorage = multer.memoryStorage();
const uploadMemory = multer({ storage: memoryStorage });

// ===== CORE AUXILIARY DISPATCH EMAIL LOGIC =====
async function sendSystemNotificationEmail(to, subject, text, html) {
    if (!process.env.BREVO_API_KEY || !process.env.EMAIL_USER) {
        console.warn("⚠ Notification system idling: Credentials missing or empty.");
        return;
    }

    try {
        const sendSmtpEmail = new Brevo.SendSmtpEmail();
        sendSmtpEmail.subject = subject;
        sendSmtpEmail.htmlContent = html || `

${text}</p>`;
        sendSmtpEmail.sender = { name: "DAN74TECH MEDIA", email:
process.env.EMAIL_USER };
        sendSmtpEmail.to = [{ email: to }];

        await brevoEmailInstance.sendTransacEmail(sendSmtpEmail);
        console.log("✉ System alert dispatched successfully to: ${to}");
    } catch (err) {
        console.error("❌ Notification Email Dispatch Fault:", err);
    }
}

// ===== ARCHITECTURAL PROTECTION SECURITY MIDDLEWARE =====
const verifySystemToken = (req, res, next) => {
    const authHeader = req.headers['authorization'];
    const token = authHeader && authHeader.split(' ')[1];

    if (!token) return res.status(401).json({ error: "Access token validation mapping empty.
Unauthorized." });

    jwt.verify(token, JWT_SECRET, (err, user) => {
        if (err) return res.status(403).json({ error: "Token reference structural compromise detected.
Forbidden." });
        req.user = user;
        next();
    });
};


```

```

};

const verifyAdminAccess = (req, res, next) => {
  verifySystemToken(req, res, () => {
    if (!req.user || req.user.role !== 'admin') {
      return res.status(403).json({ error: "Console Operation Limited to System Administrators Only." });
    }
    next();
  });
};

// ===== MODULE 1: AUTHENTICATION & SECURITY DATA ENGINE =====

app.post('/api/auth/register', async (req, res) => {
  try {
    const { name, email, password, role, phone } = req.body;
    const userRole = role || 'client';

    const checkUser = await pool.query('SELECT id FROM users WHERE email = $1 AND is_deleted = FALSE', [email]);
    if (checkUser.rows.length > 0) {
      return res.status(400).json({ error: "User registration email conflict detected." });
    }

    const salt = await bcrypt.genSalt(10);
    const hashedPassword = await bcrypt.hash(password, salt);

    const result = await pool.query(
      `INSERT INTO users (name, email, password, role, phone, is_deleted)
      VALUES ($1, $2, $3, $4, $5, FALSE)
      RETURNING id, name, email, role, phone`,
      [name, email, hashedPassword, userRole, phone]
    );

    const userNode = result.rows[0];
    const accessToken = jwt.sign({ id: userNode.id, email: userNode.email, role: userNode.role }, JWT_SECRET, { expiresIn: '24h' });

    await sendSystemNotificationEmail(
      userNode.email,
      "Welcome to DAN74TECH MEDIA",
      `Hello ${userNode.name}, your workspace engine profile setup is successfully validated.`
    );

    res.json({ success: true, token: accessToken, user: userNode });
  }
});

```

```

    } catch (err) {
      res.status(500).json({ error: err.message });
    }
  });

app.post('/api/auth/login', async (req, res) => {
  try {
    const { email, password } = req.body;

    const result = await pool.query('SELECT * FROM users WHERE email = $1 AND
is_deleted = FALSE', [email]);
    if (result.rows.length === 0) {
      return res.status(401).json({ success: false, message: "Invalid configuration options
submitted" });
    }

    const user = result.rows[0];
    let isMatch = false;

    if (user.password.startsWith('$2a$') || user.password.startsWith('$2b$')) {
      isMatch = await bcrypt.compare(password, user.password);
    } else {
      isMatch = (user.password === password);
    }

    if (!isMatch) {
      return res.status(401).json({ success: false, message: "Invalid configuration options
submitted" });
    }

    const accessToken = jwt.sign({ id: user.id, email: user.email, role: user.role },
JWT_SECRET, { expiresIn: '24h' });

    res.json({
      success: true,
      token: accessToken,
      user: { id: user.id, name: user.name, email: user.email, role: user.role, phone: user.phone
    }
    });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// ===== MODULE 2: USERS MANAGEMENT INTERFACE
=====

app.get('/api/users', verifyAdminAccess, async (req, res) => {

```

```

    try {
      const result = await pool.query(
        'SELECT id, name, email, role, phone, created_at FROM users WHERE is_deleted =
FALSE ORDER BY id DESC'
      );
      res.json(result.rows);
    } catch (err) {
      res.status(500).json({ error: err.message });
    }
  });

app.get('/api/users/:id', verifyAdminAccess, async (req, res) => {
  try {
    const result = await pool.query(
      'SELECT id, name, email, role, phone, created_at FROM users WHERE id = $1 AND
is_deleted = FALSE',
      [req.params.id]
    );
    if (result.rows.length === 0) {
      return res.status(404).json({ error: 'User not found' });
    }
    res.json(result.rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.put('/api/users/:id', verifyAdminAccess, async (req, res) => {
  try {
    const { name, email, role, phone, password } = req.body;
    let query;
    let values;

    if (password) {
      const hashedPassword = await bcrypt.hash(password, 10);
      query = `UPDATE users SET name=$1, email=$2, role=$3, phone=$4, password=$5,
updated_at=CURRENT_TIMESTAMP WHERE id=$6 RETURNING id,name,email,role,phone`;
      values = [name, email, role, phone, hashedPassword, req.params.id];
    } else {
      query = `UPDATE users SET name=$1, email=$2, role=$3, phone=$4,
updated_at=CURRENT_TIMESTAMP WHERE id=$5 RETURNING id,name,email,role,phone`;
      values = [name, email, role, phone, req.params.id];
    }

    const result = await pool.query(query, values);
    res.json({ success: true, data: result.rows[0] });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

```
}  
});
```

```
app.delete('/api/users/:id', verifyAdminAccess, async (req, res) => {  
  try {  
    await pool.query('UPDATE users SET is_deleted = TRUE, updated_at =  
CURRENT_TIMESTAMP WHERE id = $1', [req.params.id]);  
    res.json({ success: true, message: "User credentials safely deactivated." });  
  } catch (err) {  
    res.status(500).json({ error: err.message });  
  }  
});
```

```
// ===== MODULE 2 (B): SERVICES MANAGEMENT INTERFACE  
=====
```

```
app.get('/api/services', async (req, res) => {  
  try {  
    const data = await pool.query('SELECT * FROM services WHERE is_deleted = FALSE  
ORDER BY id DESC');  
    res.json(data.rows);  
  } catch (err) {  
    res.status(500).json({ error: err.message });  
  }  
});
```

```
app.post('/api/services', verifyAdminAccess, async (req, res) => {  
  try {  
    const { name, icon, description } = req.body;  
    const result = await pool.query(  
      `INSERT INTO services (name, icon, description) VALUES ($1, $2, $3) RETURNING *`,  
      [name, icon, description]  
    );  
    res.json({ success: true, data: result.rows[0] });  
  } catch (err) {  
    res.status(500).json({ error: err.message });  
  }  
});
```

```
app.put('/api/services/:id', verifyAdminAccess, async (req, res) => {  
  try {  
    const { id } = req.params;  
    const { name, icon, description } = req.body;  
    const result = await pool.query(  
      `UPDATE services SET name = $1, icon = $2, description = $3, updated_at =  
CURRENT_TIMESTAMP WHERE id = $4 RETURNING *`,  
      [name, icon, description, id]  
    );  
  }  
});
```

```

        res.json({ success: true, data: result.rows[0] });
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

app.delete('/api/services/:id', verifyAdminAccess, async (req, res) => {
    try {
        await pool.query('UPDATE services SET is_deleted = TRUE, updated_at =
CURRENT_TIMESTAMP WHERE id = $1', [req.params.id]);
        res.json({ success: true });
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

```

// ===== MODULE 3: SUB-SERVICES & PRICING ENGINE
=====

```

app.get('/api/sub-services', async (req, res) => {
    try {
        const query = `
            SELECT sub_services.*, services.name AS category_name
            FROM sub_services
            JOIN services ON sub_services.service_id = services.id
            WHERE sub_services.is_deleted = FALSE AND services.is_deleted = FALSE
            ORDER BY sub_services.id DESC
        `;
        const data = await pool.query(query);
        res.json(data.rows);
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

```

```

app.get('/api/sub-services/:serviceName', async (req, res) => {
    try {
        const serviceName = req.params.serviceName.replace(/-/g, ' ').toLowerCase();
        const result = await pool.query(`
            SELECT ss.*
            FROM sub_services ss
            JOIN services s ON ss.service_id = s.id
            WHERE LOWER(s.name) = $1 AND ss.is_deleted = FALSE
            ORDER BY ss.id ASC
        `, [serviceName]);
        res.json(result.rows);
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

```

```

    }
  });

app.post('/api/sub-services', verifyAdminAccess, async (req, res) => {
  try {
    const { service_id, name, price, description } = req.body;
    const result = await pool.query(
      `INSERT INTO sub_services (service_id, name, price, description) VALUES ($1, $2,
      COALESCE($3, 0.00), $4) RETURNING *`,
      [service_id, name, price, description]
    );
    res.json({ success: true, data: result.rows[0] });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

```

app.delete('/api/sub-services/:id', verifyAdminAccess, async (req, res) => {
  try {
    await pool.query('UPDATE sub_services SET is_deleted = TRUE, updated_at =
    CURRENT_TIMESTAMP WHERE id = $1', [req.params.id]);
    res.json({ success: true });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

```

// ===== MODULE 4: OPERATIONS & ORDER FLOW ENGINE
=====

```

```

// Advanced Query Builder
const buildAdvancedQuery = (tableName, queryParams, searchableColumns = []) => {
  let { page = 1, limit = 100, sort = 'id', order = 'DESC', search = "", ...filters } = queryParams;

  let whereClauses = ['is_deleted = FALSE'];
  let values = [];
  let valueIndex = 1;

  if (search && searchableColumns.length > 0) {
    const searchClauses = searchableColumns.map(col => `${col} ILIKE ${`${valueIndex}`}`);
    whereClauses.push(`${searchClauses.join(' OR ')}`);
    values.push(`%${search}%`);
    valueIndex++;
  }

  for (const [key, val] of Object.entries(filters)) {
    if (val !== undefined && val !== "") {
      whereClauses.push(`${key} = ${`${valueIndex}`}`);
    }
  }
}

```

```

        values.push(val);
        valueIndex++;
    }
}

const whereString = whereClauses.length > 0 ? `WHERE ${whereClauses.join(' AND ')}` : "";
const offset = (parseInt(page) - 1) * parseInt(limit);

const safeSort = sort.replace(/[^a-zA-Z0-9_]/g, "");
const safeOrder = order.toUpperCase() === 'ASC' ? 'ASC' : 'DESC';

const queryString = `SELECT * FROM ${tableName} ${whereString} ORDER BY ${safeSort}
${safeOrder} LIMIT ${parseInt(limit)} OFFSET ${offset}`;
const countQuery = `SELECT COUNT(*) FROM ${tableName} ${whereString}`;

return { queryString, countQuery, values };
};

app.get('/api/orders', verifyAdminAccess, async (req, res) => {
    try {
        const searchableFields = ['customer_name', 'email', 'phone', 'service', 'invoice_number'];
        const { queryString, countQuery, values } = buildAdvancedQuery('orders', req.query,
searchableFields);

        const data = await pool.query(queryString, values);
        const countData = await pool.query(countQuery, values);

        res.json({
            data: data.rows,
            meta: {
                total: parseInt(countData.rows[0].count),
                page: parseInt(req.query.page || 1),
                limit: parseInt(req.query.limit || 100)
            }
        });
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

app.get('/api/orders/user/:userId', verifySystemToken, async (req, res) => {
    try {
        const result = await pool.query('SELECT * FROM orders WHERE user_id = $1 AND
is_deleted = FALSE ORDER BY id DESC', [req.params.userId]);
        res.json(result.rows);
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
}

```

```

});

app.post('/api/orders', async (req, res) => {
  try {
    const {
      user_id, customer_name, phone, service, sub_service,
      domain, device_model, project_details, price, status, payment_status
    } = req.body;

    const result = await pool.query(
      `INSERT INTO orders (
        user_id, customer_name, phone, service, sub_service, domain,
        device_model, project_details, price, status, payment_status
      ) VALUES (
        $1, $2, $3, $4, $5,
        COALESCE($6, 'N/A'),
        COALESCE($7, 'Web Client'),
        $8,
        COALESCE($9, 0.00),
        COALESCE($10, 'pending'),
        COALESCE($11, 'unpaid')
      ) RETURNING *`,
      [
        user_id || null, customer_name, phone, service, sub_service || null,
        domain || null, device_model || null, project_details || null,
        price || null, status || null, payment_status || null
      ]
    );

    const orderData = result.rows[0];

    if (user_id) {
      await pool.query(
        `INSERT INTO notifications (user_id, title, message, channel, status) VALUES ($1,
        $2, $3, 'dashboard', 'unread')`,
        [user_id, 'Order Received Successfully', `Your order sequence entry for ${service} is
        submitted.`]
      );
    }

    await sendSystemNotificationEmail(
      process.env.ADMIN_EMAIL || 'dan74techmedia@gmail.com',
      `🔴 NEW SERVICE ORDER: ${service}`,
      `A new deployment sequence has been raised by ${customer_name}. Phone: ${phone}.
      Package details: ${sub_service}.`,
      `

### New Production Order Raised



<strong>Client:</strong>
      ${customer_name}</p><p><strong>Module:</strong> ${service}
      (${sub_service})</p><p><strong>Scope:</strong> ${project_details || 'None specified'}</p>`
    );
  }
});


```

```

    );

    res.json({ success: true, data: orderData });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.put('/api/orders/:id', verifyAdminAccess, async (req, res) => {
  try {
    const { id } = req.params;
    const { status, payment_status, price } = req.body;
    const result = await pool.query(
      `UPDATE orders SET status = COALESCE($1, status), payment_status =
      COALESCE($2, payment_status), price = COALESCE($3, price), updated_at =
      CURRENT_TIMESTAMP WHERE id = $4 RETURNING *`,
      [status, payment_status, price, id]
    );
    res.json({ success: true, data: result.rows[0] });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.delete('/api/orders/:id', verifyAdminAccess, async (req, res) => {
  try {
    await pool.query('UPDATE orders SET is_deleted = TRUE, updated_at =
    CURRENT_TIMESTAMP WHERE id = $1', [req.params.id]);
    res.json({ success: true });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// ===== MODULE 5: PORTFOLIO & WORKFLOW STATUS BAR ENGINE
// =====

app.get('/api/portfolio', async (req, res) => {
  try {
    const data = await pool.query('SELECT * FROM portfolio WHERE is_deleted = FALSE
    ORDER BY id DESC');
    res.json(data.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.post('/api/portfolio', verifyAdminAccess, uploadLocal.single('image'), async (req, res) => {

```

```

    try {
      const { title, category, description, order_id, progress, status } = req.body;
      let link = req.body.link || "";
      if (req.file) {
        link = `/uploads/${req.file.filename}`;
      }
      const result = await pool.query(
        `INSERT INTO portfolio (title, category, description, link, order_id, progress, status)
        VALUES ($1, $2, $3, $4, $5, COALESCE($6, 0), COALESCE($7, 'In Progress'))
RETURNING *`,
        [title, category, description, link, order_id || null, progress, status]
      );
      res.json({ success: true, data: result.rows[0] });
    } catch (err) {
      res.status(500).json({ error: err.message });
    }
  });

  app.put('/api/portfolio/:id', verifyAdminAccess, async (req, res) => {
    try {
      const { id } = req.params;
      const { progress, status, title, description } = req.body;
      const result = await pool.query(
        `UPDATE portfolio SET progress = COALESCE($1, progress), status = COALESCE($2,
status), title = COALESCE($3, title), description = COALESCE($4, description), updated_at =
CURRENT_TIMESTAMP WHERE id = $5 RETURNING *`,
        [progress, status, title, description, id]
      );
      res.json({ success: true, data: result.rows[0] });
    } catch (err) {
      res.status(500).json({ error: err.message });
    }
  });

  app.delete('/api/portfolio/:id', verifyAdminAccess, async (req, res) => {
    try {
      await pool.query('UPDATE portfolio SET is_deleted = TRUE, updated_at =
CURRENT_TIMESTAMP WHERE id = $1', [req.params.id]);
      res.json({ success: true });
    } catch (err) {
      res.status(500).json({ error: err.message });
    }
  });

  // ===== MODULE 6: CASE STUDIES HUB =====

  app.get('/api/case-studies', async (req, res) => {
    try {

```

```

    const result = await pool.query('SELECT * FROM case_studies WHERE is_deleted =
FALSE ORDER BY id DESC');
    res.json(result.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.get('/api/case-studies/:id', async (req, res) => {
  try {
    const result = await pool.query(
      'SELECT * FROM case_studies WHERE id = $1 AND is_deleted = FALSE',
      [req.params.id]
    );
    if (result.rows.length === 0) return res.status(404).json({ error: 'Case study not found' });
    res.json(result.rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.put('/api/case-studies/:id', verifyAdminAccess, async (req, res) => {
  try {
    const { title, category, challenge, solution, result, image_url } = req.body;
    const updated = await pool.query(
      `UPDATE case_studies SET title=$1, category=$2, challenge=$3, solution=$4,
result=$5, image_url=$6, updated_at=CURRENT_TIMESTAMP WHERE id=$7 RETURNING *`,
      [title, category, challenge, solution, result, image_url, req.params.id]
    );
    res.json({ success: true, data: updated.rows[0] });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.post('/api/case-studies', verifyAdminAccess, async (req, res) => {
  try {
    const { title, category, challenge, solution, result, image_url } = req.body;
    const out = await pool.query(
      `INSERT INTO case_studies (title, category, challenge, solution, result, image_url)
VALUES ($1, $2, $3, $4, $5, $6) RETURNING *`,
      [title, category, challenge, solution, result, image_url]
    );
    res.json({ success: true, data: out.rows[0] });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

```

app.delete('/api/case-studies/:id', verifyAdminAccess, async (req, res) => {
  try {
    await pool.query('UPDATE case_studies SET is_deleted = TRUE, updated_at =
CURRENT_TIMESTAMP WHERE id = $1', [req.params.id]);
    res.json({ success: true });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// ===== MODULE 7: REPUTATION & TESTIMONIAL AUDITING MATRIX
=====

app.get('/api/testimonials/approved', async (req, res) => {
  try {
    const data = await pool.query("SELECT * FROM testimonials WHERE status = 'approved'
AND is_deleted = FALSE ORDER BY id DESC");
    res.json(data.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.get('/api/testimonials', async (req, res) => {
  try {
    const { status } = req.query;
    let queryStr = 'SELECT * FROM testimonials WHERE is_deleted = FALSE ORDER BY id
DESC';
    let params = [];

    if (status === 'approved') {
      queryStr = "SELECT * FROM testimonials WHERE status = 'approved' AND is_deleted
= FALSE ORDER BY id DESC";
    }

    const data = await pool.query(queryStr, params);
    res.json(data.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.post('/api/testimonials', async (req, res) => {
  try {
    const { client_name, company, rating, review } = req.body;
    const result = await pool.query(
      `INSERT INTO testimonials (client_name, company, rating, review, status) VALUES ($1,`

```

```

    $2, $3, $4, 'pending') RETURNING *,
      [client_name, company, rating, review]
    );
    res.json({ success: true, data: result.rows[0] });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

```

// Upgraded specific feature toggle handler matching database script
app.put('/api/testimonials/:id', verifyAdminAccess, async (req, res) => {
  try {
    const { status, is_featured } = req.body;
    const result = await pool.query(
      `UPDATE testimonials
      SET status = COALESCE($1, status), is_featured = COALESCE($2, is_featured),
      updated_at = CURRENT_TIMESTAMP
      WHERE id = $3 RETURNING *`,
      [status, is_featured, req.params.id]
    );
    res.json({ success: true, data: result.rows[0] });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

```

// Legacy handler retained for frontend button backward compatibility
app.put('/api/testimonials/:id/approve', verifyAdminAccess, async (req, res) => {
  try {
    const result = await pool.query(`UPDATE testimonials SET status = 'approved', updated_at = CURRENT_TIMESTAMP WHERE id = $1 RETURNING *`, [req.params.id]);
    res.json({ success: true, data: result.rows[0] });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

```

app.delete('/api/testimonials/:id', verifyAdminAccess, async (req, res) => {
  try {
    await pool.query('UPDATE testimonials SET is_deleted = TRUE, updated_at = CURRENT_TIMESTAMP WHERE id = $1', [req.params.id]);
    res.json({ success: true });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

// ===== MODULE 8: BLOG CONSOLE & SEARCH ENGINE DATA ROUTING

=====

```
const getBlogPosts = async (req, res) => {
  try {
    const result = await pool.query('SELECT * FROM blog_posts WHERE is_deleted = FALSE
ORDER BY id DESC');
    res.json(result.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};
app.get('/api/blog', getBlogPosts);
app.get('/api/blogs', getBlogPosts);

app.get('/api/blog/id/:id', async (req, res) => {
  try {
    const result = await pool.query('SELECT * FROM blog_posts WHERE id = $1 AND
is_deleted = FALSE', [req.params.id]);
    if (result.rows.length === 0) return res.status(404).json({ error: 'Blog post not found' });
    res.json(result.rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

const updateBlogPost = async (req, res) => {
  try {
    const { title, category, content, summary, image_url, slug, seo_title, seo_description } =
req.body;
    const result = await pool.query(
      `UPDATE blog_posts SET title=$1, category=$2, content=$3, summary=$4,
image_url=$5, slug=$6, seo_title=$7, seo_description=$8,
updated_at=CURRENT_TIMESTAMP WHERE id=$9 RETURNING *`,
      [title, category, content, summary, image_url, slug, seo_title, seo_description,
req.params.id]
    );
    res.json({ success: true, data: result.rows[0] });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};
app.put('/api/blog/:id', verifyAdminAccess, updateBlogPost);
app.put('/api/blogs/:id', verifyAdminAccess, updateBlogPost);

app.get('/api/blog/search', async (req, res) => {
  try {
    const { q } = req.query;
    if (!q) {
```

```

        const result = await pool.query('SELECT * FROM blog_posts WHERE is_deleted =
FALSE ORDER BY id DESC');
        return res.json(result.rows);
    }
    const result = await pool.query(
        `SELECT * FROM blog_posts WHERE (LOWER(title) LIKE $1 OR LOWER(content)
LIKE $1) AND is_deleted = FALSE ORDER BY id DESC`,
        [`%${q.toLowerCase()}%`]
    );
    res.json(result.rows);
} catch (err) {
    res.status(500).json({ error: err.message });
}
});

app.get('/api/blog/:slug', async (req, res) => {
    try {
        const result = await pool.query('SELECT * FROM blog_posts WHERE slug = $1 AND
is_deleted = FALSE', [req.params.slug]);
        if (result.rows.length === 0) return res.status(404).json({ error: "Article node reference
absent" });
        res.json(result.rows[0]);
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

const createBlogPost = async (req, res) => {
    try {
        const { title, category, content, summary, image_url, slug, seo_title, seo_description } =
req.body;
        const result = await pool.query(
            `INSERT INTO blog_posts (title, category, content, summary, image_url, slug, seo_title,
seo_description)
VALUES ($1, $2, $3, $4, $5, $6, COALESCE($7, $1), COALESCE($8, $4))
RETURNING *`,
            [title, category, content, summary, image_url, slug, seo_title, seo_description]
        );
        res.json({ success: true, data: result.rows[0] });
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
};

app.post('/api/blog', verifyAdminAccess, createBlogPost);
app.post('/api/blogs', verifyAdminAccess, createBlogPost);

app.delete('/api/blog/:id', verifyAdminAccess, async (req, res) => {
    try {

```

```

    await pool.query('UPDATE blog_posts SET is_deleted = TRUE, updated_at =
CURRENT_TIMESTAMP WHERE id = $1', [req.params.id]);
    res.json({ success: true });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

// ===== MODULE 9: NEWSLETTER AUDIENCE SUBSCRIPTION REGISTER
=====

```

app.post('/api/subscribers', async (req, res) => {
  try {
    const { email } = req.body;
    const result = await pool.query(
      `INSERT INTO subscribers (email, status)
      VALUES ($1, 'active')
      ON CONFLICT (email)
      DO UPDATE SET status = 'active', is_deleted = FALSE
      RETURNING *`,
      [email]
    );
    res.json({ success: true, data: result.rows[0] });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

```

app.get('/api/subscribers', verifyAdminAccess, async (req, res) => {
  try {
    const result = await pool.query('SELECT * FROM subscribers WHERE is_deleted = FALSE
ORDER BY id DESC');
    res.json(result.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

```

app.post('/api/subscribers/broadcast', verifyAdminAccess, async (req, res) => {
  const { subject, message, html } = req.body;

  if (!subject || (!message && !html)) {
    return res.status(400).json({ error: 'Missing subject or content data body parameters.' });
  }

  try {
    const dbResult = await pool.query("SELECT email FROM subscribers WHERE status =
'active' AND is_deleted = FALSE");

```

```

const subscribers = dbResult.rows.map(row => row.email);

if (subscribers.length === 0) {
  return res.status(200).json({ sent: 0, failed: 0, message: 'No active subscription lines
detected.' });
}

const formattedRecipients = subscribers.map(email => ({ email: email }));
const sendSmtpEmail = new Brevo.SendSmtpEmail();
sendSmtpEmail.subject = subject;
sendSmtpEmail.htmlContent = html || `
```

```

    } catch (err) {
      res.status(500).json({ error: err.message });
    }
  });

app.post('/api/media/upload', verifyAdminAccess, uploadMemory.single('file'), async (req, res)
=> {
  try {
    if (!req.file) return res.status(400).json({ error: "Required file node absent" });

    const uploadStream = cloudinary.uploader.upload_stream(
      { folder: "dan74tech_media_library" },
      async (error, result) => {
        if (error) return res.status(500).json({ error: error.message });

        const savedAsset = await pool.query(
          `INSERT INTO media_library (public_id, url, filename, format, bytes) VALUES ($1,
$2, $3, $4, $5) RETURNING *`,
          [result.public_id, result.secure_url, req.file.originalname, result.format, result.bytes]
        );
        res.json({ success: true, data: savedAsset.rows[0] });
      }
    );
    streamifier.createReadStream(req.file.buffer).pipe(uploadStream);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

```

app.delete('/api/media/:id', verifyAdminAccess, async (req, res) => {
  try {
    const target = await pool.query('SELECT * FROM media_library WHERE id = $1 AND
is_deleted = FALSE', [req.params.id]);
    if (target.rows.length === 0) return res.status(404).json({ error: "Target data row reference
absent" });

```

```

    const asset = target.rows[0];
    await cloudinary.uploader.destroy(asset.public_id);
    await pool.query('UPDATE media_library SET is_deleted = TRUE, updated_at =
CURRENT_TIMESTAMP WHERE id = $1', [req.params.id]);

    res.json({ success: true });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

// ===== MODULE 11: FINANCIAL LOGISTICS & INVOICING

=====

```
app.get('/api/invoices', verifyAdminAccess, async (req, res) => {
  try {
    const result = await pool.query('SELECT * FROM invoices WHERE is_deleted = FALSE
ORDER BY id DESC');
    res.json(result.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

```
app.post('/api/invoices', verifyAdminAccess, async (req, res) => {
  try {
    const { order_id, invoice_number, client_name, client_email, amount, pdf_url, status } =
req.body;
    const result = await pool.query(
      `INSERT INTO invoices (order_id, invoice_number, client_name, client_email, amount,
pdf_url, status)
VALUES ($1, $2, $3, $4, $5, $6, COALESCE($7, 'unpaid')) RETURNING *`,
      [order_id, invoice_number, client_name, client_email, amount, pdf_url, status]
    );
    res.json({ success: true, data: result.rows[0] });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

```
app.get('/api/invoices/:id/download', async (req, res) => {
  try {
    const target = await pool.query('SELECT * FROM invoices WHERE id = $1 AND
is_deleted = FALSE', [req.params.id]);
    if (target.rows.length === 0) return res.status(404).json({ error: "Invoice sequence
mismatch." });
    const inv = target.rows[0];

    const doc = new PDFDocument({ margin: 50 });
    res.setHeader('Content-Type', 'application/pdf');
    res.setHeader('Content-Disposition', `attachment;
filename=Invoice-${inv.invoice_number}.pdf`);
    doc.pipe(res);

    doc.fillColor('#0044ff').fontSize(24).text('DAN74TECH MEDIA', { underline: true });
    doc.fillColor('#1a1a1a').fontSize(10).text('Professional Portal & Technology Hub', { align:
'left' }).moveDown(2);

    doc.fontSize(14).text(`INVOICE STATEMENT: #${inv.invoice_number}`, { stroke: false });
    doc.text(`Issued To: ${inv.client_name}`);
```

```

    doc.text(`Contact Point: ${inv.client_email || 'N/A'}`).moveDown(1);

    doc.rect(50, doc.y, 500, 2).fill('#ff2a2a').moveDown(1);

    doc.fillColor('#1a1a1a').fontSize(12).text(`Billed Total: $$${inv.amount}`, { bold: true });
    doc.text(`Status Manifest: ${inv.status.toUpperCase()}`).moveDown(2);

    doc.fontSize(10).fillColor('#666666').text('Thank you for partnering with DAN74TECH
MEDIA. For queries, contact support via WhatsApp routing loops.', { italic: true });
    doc.end();
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.delete('/api/invoices/:id', verifyAdminAccess, async (req, res) => {
  try {
    await pool.query('UPDATE invoices SET is_deleted = TRUE, updated_at =
CURRENT_TIMESTAMP WHERE id = $1', [req.params.id]);
    res.json({ success: true });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// ===== MODULE 12: REAL-TIME NOTIFICATION DISPATCH ENGINE
=====

app.get('/api/notifications/user/:userId', verifySystemToken, async (req, res) => {
  try {
    const result = await pool.query('SELECT * FROM notifications WHERE user_id = $1 AND
is_deleted = FALSE ORDER BY id DESC', [req.params.userId]);
    res.json(result.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.post('/api/notifications', verifyAdminAccess, async (req, res) => {
  try {
    const { user_id, title, message, channel } = req.body;
    const result = await pool.query(
      `INSERT INTO notifications (user_id, title, message, channel, status) VALUES ($1, $2,
$3, COALESCE($4, 'dashboard'), 'unread') RETURNING *`,
      [user_id, title, message, channel]
    );
    res.json({ success: true, data: result.rows[0] });
  } catch (err) {

```

```

        res.status(500).json({ error: err.message });
    }
});

// ===== MODULE 13: SUPPORT TICKETING ROUTER =====

app.get('/api/tickets', verifyAdminAccess, async (req, res) => {
    try {
        const result = await pool.query('SELECT * FROM support_tickets WHERE is_deleted = FALSE ORDER BY id DESC');
        res.json(result.rows);
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

app.post('/api/tickets', verifySystemToken, async (req, res) => {
    try {
        const { user_id, subject, description, priority } = req.body;
        const result = await pool.query(
            `INSERT INTO support_tickets (user_id, subject, description, priority, status) VALUES ($1, $2, $3, COALESCE($4, 'medium'), 'open') RETURNING *`,
            [user_id, subject, description, priority]
        );
        res.json({ success: true, data: result.rows[0] });
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

app.put('/api/tickets/:id', verifyAdminAccess, async (req, res) => {
    try {
        const { id } = req.params;
        const { status, priority } = req.body;
        const result = await pool.query(
            `UPDATE support_tickets SET status = COALESCE($1, status), priority = COALESCE($2, priority), updated_at = CURRENT_TIMESTAMP WHERE id = $3 RETURNING *`,
            [status, priority, id]
        );
        res.json({ success: true, data: result.rows[0] });
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

app.delete('/api/tickets/:id', verifyAdminAccess, async (req, res) => {
    try {

```

```

        await pool.query('UPDATE support_tickets SET is_deleted = TRUE, updated_at =
CURRENT_TIMESTAMP WHERE id = $1', [req.params.id]);
        res.json({ success: true });
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

// ===== MODULE 14: MESSAGES HUB (CONTACT FORM LOGISTICS)
=====

app.get('/api/messages', verifyAdminAccess, async (req, res) => {
    try {
        const result = await pool.query('SELECT * FROM messages WHERE is_deleted = FALSE
ORDER BY id DESC');
        res.json(result.rows);
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

app.post('/api/messages', async (req, res) => {
    try {
        const { name, email, subject, message } = req.body;
        const result = await pool.query(
            `INSERT INTO messages (name, email, subject, message, status) VALUES ($1, $2,
COALESCE($3, 'General Inquiry'), $4, 'unread') RETURNING *`,
            [name, email, subject, message]
        );
        res.json({ success: true, data: result.rows[0] });
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

app.delete('/api/messages/:id', verifyAdminAccess, async (req, res) => {
    try {
        await pool.query('UPDATE messages SET is_deleted = TRUE, updated_at =
CURRENT_TIMESTAMP WHERE id = $1', [req.params.id]);
        res.json({ success: true });
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

// ===== MODULE 15: APPOINTMENTS & CONSULTATION OPERATIONS
=====

```

```

app.get('/api/consultations', verifyAdminAccess, async (req, res) => {
  try {
    const result = await pool.query('SELECT * FROM consultations WHERE is_deleted =
FALSE ORDER BY id DESC');
    res.json(result.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.post('/api/consultations', async (req, res) => {
  try {
    const { client_name, client_email, booking_date, booking_time, platform } = req.body;
    const result = await pool.query(
      `INSERT INTO consultations (client_name, client_email, booking_date, booking_time,
platform, status)
VALUES ($1, $2, $3, $4, COALESCE($5, 'Google Meet'), 'pending') RETURNING *`,
      [client_name, client_email, booking_date, booking_time, platform]
    );
    res.json({ success: true, data: result.rows[0] });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.put('/api/consultations/:id', verifyAdminAccess, async (req, res) => {
  try {
    const { status } = req.body;
    const result = await pool.query(
      `UPDATE consultations SET status = $1, updated_at = CURRENT_TIMESTAMP
WHERE id = $2 RETURNING *`,
      [status, req.params.id]
    );
    res.json({ success: true, data: result.rows[0] });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.delete('/api/consultations/:id', verifyAdminAccess, async (req, res) => {
  try {
    await pool.query('UPDATE consultations SET is_deleted = TRUE, updated_at =
CURRENT_TIMESTAMP WHERE id = $1', [req.params.id]);
    res.json({ success: true });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

```
// ===== MODULE 16: FILE DELIVERY NETWORK LAYER
=====
```

```
app.get('/api/file-deliveries', verifyAdminAccess, async (req, res) => {
  try {
    const result = await pool.query('SELECT * FROM file_deliveries WHERE is_deleted =
FALSE ORDER BY id DESC');
    res.json(result.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

```
app.get('/api/file-deliveries/client/:clientId', verifySystemToken, async (req, res) => {
  try {
    const result = await pool.query('SELECT * FROM file_deliveries WHERE client_id = $1
AND is_deleted = FALSE ORDER BY id DESC', [req.params.clientId]);
    res.json(result.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

```
app.post('/api/file-deliveries', verifyAdminAccess, uploadLocal.single('attachment'), async (req,
res) => {
  try {
    const { order_id, client_id, file_name, file_url, expiry_date, status } = req.body;
    let finalUrl = file_url || "";
    let finalName = file_name || 'Delivery Asset File';

    if (req.file) {
      finalUrl = `/uploads/${req.file.filename}`;
      finalName = req.file.originalname;
    }

    const result = await pool.query(
      `INSERT INTO file_deliveries (order_id, client_id, file_name, file_url, expiry_date, status)
VALUES ($1, $2, $3, $4, $5, COALESCE($6, 'available')) RETURNING *`,
      [order_id || null, client_id || null, finalName, finalUrl, expiry_date || null, status]
    );
    res.json({ success: true, data: result.rows[0] });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

```
app.delete('/api/file-deliveries/:id', verifyAdminAccess, async (req, res) => {
```

```

    try {
      await pool.query('UPDATE file_deliveries SET is_deleted = TRUE, updated_at =
CURRENT_TIMESTAMP WHERE id = $1', [req.params.id]);
      res.json({ success: true });
    } catch (err) {
      res.status(500).json({ error: err.message });
    }
  });

```

// ===== MODULE 17: ADMINISTRATIVE STATISTICS METRICS
DASHBOARD ENGINE =====

```

app.get('/api/admin/stats', verifyAdminAccess, async (req, res) => {
  try {
    const userCount = await pool.query('SELECT COUNT(*) FROM users WHERE is_deleted
= FALSE');
    const orderCount = await pool.query('SELECT COUNT(*) FROM orders WHERE
is_deleted = FALSE');
    const pendingCount = await pool.query("SELECT COUNT(*) FROM orders WHERE status
= 'pending' AND is_deleted = FALSE");
    const ticketCount = await pool.query("SELECT COUNT(*) FROM support_tickets WHERE
status = 'open' AND is_deleted = FALSE");
    const earningsSum = await pool.query("SELECT SUM(price) FROM orders WHERE
payment_status = 'paid' AND is_deleted = FALSE");
    const consultationCount = await pool.query("SELECT COUNT(*) FROM consultations
WHERE status = 'pending' AND is_deleted = FALSE");

    res.json({
      users: parseInt(userCount.rows[0].count || '0'),
      orders: parseInt(orderCount.rows[0].count || '0'),
      pending_orders: parseInt(pendingCount.rows[0].count || '0'),
      open_tickets: parseInt(ticketCount.rows[0].count || '0'),
      total_earnings: parseFloat(earningsSum.rows[0].sum || '0.00'),
      pending_consultations: parseInt(consultationCount.rows[0].count || '0')
    });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

// ===== UNIVERSAL BULK ACTION & CORE FETCH ENGINE
=====

```

app.put('/api/:table/:id/status', verifyAdminAccess, async (req, res) => {
  const { is_deleted } = req.body;
  try {
    const table = req.params.table.replace(/^[^a-z_]/g, "");
    const result = await pool.query(

```

```

        `UPDATE ${table} SET is_deleted = $1, updated_at = CURRENT_TIMESTAMP WHERE
id = $2 RETURNING *`,
        [is_deleted, req.params.id]
    );
    res.json({ success: true, data: result.rows[0] });
} catch (err) {
    res.status(500).json({ error: err.message });
}
});

```

```

app.post('/api/:table/bulk', verifyAdminAccess, async (req, res) => {
    const { ids, action } = req.body;
    const table = req.params.table.replace(/^[a-z_]/g, "");

    if (!Array.isArray(ids) || ids.length === 0) return res.status(400).json({ error: "No IDs provided"
});

```

```

    try {
        if (action === 'delete' || action === 'soft_delete') {
            await pool.query(`UPDATE ${table} SET is_deleted = TRUE, updated_at =
CURRENT_TIMESTAMP WHERE id = ANY($1::int[])`, [ids]);
        } else if (action === 'hard_delete') {
            await pool.query(`DELETE FROM ${table} WHERE id = ANY($1::int[])`, [ids]);
        } else if (action === 'restore') {
            await pool.query(`UPDATE ${table} SET is_deleted = FALSE, updated_at =
CURRENT_TIMESTAMP WHERE id = ANY($1::int[])`, [ids]);
        }
        res.json({ success: true, message: `Bulk ${action} executed on ${ids.length} records.` });
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

```

```

app.get('/api/:table/:id', verifySystemToken, async (req, res) => {
    const allowedTables = ['users', 'services', 'orders', 'portfolio', 'blog_posts', 'invoices',
'support_tickets', 'testimonials', 'consultations'];
    if (!allowedTables.includes(req.params.table)) return res.status(403).json({error: "Table
access forbidden."});

```

```

    try {
        const result = await pool.query(`SELECT * FROM ${req.params.table} WHERE id = $1
AND is_deleted = FALSE`, [req.params.id]);
        if (result.rows.length === 0) return res.status(404).json({error: "Record not found"});
        res.json(result.rows[0]);
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

```

```
// ===== GLOBAL APPLICATION VERIFICATION ROUTE
=====
app.get('/', (req, res) => {
  res.status(200).send('🚀 DAN74TECH MEDIA Unified Operations API Matrix is active.');
```



```
// ===== INITIALIZE SERVICE EXECUTABLE RUNTIME =====
app.listen(PORT, () => {
  console.log(`🌐 System Core Online. Framework operational on port parameter: ${PORT}`);
});
```